

Lesson 1B

Parallelism=1

This Job Graph represents your running “**Random Word Counting Job**” and shows how Flink has optimized your code into two execution vertices connected by a data shuffle boundary.

Node 1: Source: random-word-source -> FlatMap

- **What it contains:** The custom/DataGenerator source and your `.flatMap(...)` transformation (which tokenizes/maps words into `(word, 1L)` tuples).
 - **Why they are fused:** Both operators run with **Parallelism: 1** and operate element-by-element in memory without requiring data re-partitioning. Flink operator chaining combines them into a single JVM thread to eliminate serialization overhead.
 - **Metrics:**
Bytes / Records Received: 0 (because sources originate data internally rather than receiving it).
Bytes / Records Sent: 235 KB across 9,790 records emitted downstream.
-

Network Connection: HASH

- **Why HASH partitioning:** Your pipeline calls `.keyBy(tuple -> tuple.f0)`. Flink routes elements using a hash partitioner based on the key (`word`) so that all occurrences of the exact same word route to the same window instance.
 - *Even though both operators run at Parallelism 1 in this run, `.keyBy()` creates a logical key-partitioned boundary.*
-

Node 2: TumblingProcessingTimeWindows -> Sink: Print to Std. Out

- **What it contains:** The 5-second processing time tumbling window, the `.reduce(...)` sum aggregator, and the `.print()` sink.
 - **Why they are fused:** The reduce function continuously updates the window state in memory. When the 5-second timer fires, the final aggregated result for each word passes directly to the `Print` sink within the same task thread.
 - **Parallelism:** Set to **1**.
-

Key Takeaways

1. **Full Operator Fusion:** Flink collapsed your multi-step pipeline (`Source -> FlatMap -> KeyBy/Window -> Reduce -> Print`) into just **2 execution vertices** to maximize pipeline throughput.
2. **Keyed Aggregation:** The `HASH` edge guarantees that word counts are correctly grouped before entering the window reduction step.